

Weakly Supervised Cross-Modal Learning for 4D Radar Scene Flow Estimation

Jingyun Fu¹ Zhiyu Xiang^{1 2} Na Zhao³

Abstract

Due to the difficulty of obtaining ground-truth data for 4D radar scene flow estimation, previous methods typically rely on either self-supervised losses or cross-modal supervision using 3D LiDAR data, 2D images, and odometry. However, self-supervised approaches often yield suboptimal results due to radar’s inherently low-fidelity measurements, while existing cross-modal supervised methods introduce complex multi-task architecture and require costly LiDAR sensors to generate pseudo radar scene flow labels from pre-trained 3D tracking models. To overcome these limitations, we propose a task-specific iterative framework for weakly supervised radar scene flow learning, using only images and odometry for auxiliary supervision during training. Specially, we establish two novel instance-aware self-supervised losses by exploiting off-the-shelf 2D tracking and segmentation algorithms to obtain tracked instance masks, which are back-projected into 3D space to provide instance-level semantic guidance; for static regions, we integrate vehicle odometry with radar’s intrinsic motion cues to construct a rigid static loss. Extensive experiments on the real-world View-of-Delft (VoD) dataset demonstrate that our method not only surpasses state-of-the-art cross-modal supervised approaches that rely on 3D multi-object tracking on dense LiDAR point clouds but also outperforms existing fully supervised scene flow estimation methods. The code is open-sourced at <https://github.com/FuJingyun/IterFlow>.

¹Zhejiang University, Hangzhou, Zhejiang, China ²Zhejiang Provincial Key Laboratory of Multi-Modal Communication Networks and Intelligent Information Processing, Zhejiang, China ³Singapore University of Technology and Design, Singapore. Correspondence to: Zhiyu Xiang <xiangzy@zju.edu.cn>, Na Zhao <na.zhao@sutd.edu.sg>.

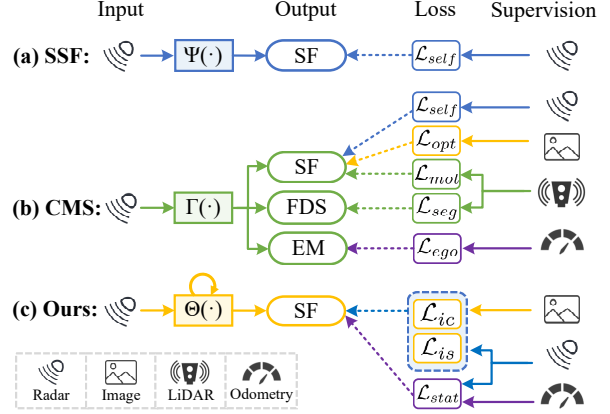


Figure 1. Comparison between existing *self-supervised* (SSF) and *cross-modal supervised* (CMS) radar scene flow estimation settings and our *weakly supervised cross-modal* learning setting.

SF, FDS, and EM denote the predicted scene flow, foreground dynamic segmentation, and ego-motion, respectively. $\mathcal{L}_{self}$ is the self-supervised losses in (Ding et al., 2022); $\mathcal{L}_{opt}$, $\mathcal{L}_{mot}$, $\mathcal{L}_{seg}$ and $\mathcal{L}_{ego}$ are cross-modal losses proposed in (Ding et al., 2023; Zhai et al., 2025; Wu et al., 2025), with supervision from 2D optical flow, 3D LiDAR-based pseudo scene flow label and FDS ground-truth, and odometry-based ego-motion. $\mathcal{L}_{ic}$ and $\mathcal{L}_{is}$ are our instance-aware losses, and $\mathcal{L}_{stat}$ is the rigid static loss.

1. Introduction

Scene flow estimation aims to predict the 3D motion field between consecutive point cloud frames, providing fine-grained, class-agnostic motion cues for 3D dynamic environment understanding. It further supports downstream tasks in autonomous driving, such as motion segmentation, visual SLAM, detection and tracking. While LiDAR-based scene flow has become mainstream (Liu et al., 2019; Puy et al., 2020; Wei et al., 2021; Kittenplon et al., 2021; Zhang et al., 2024a; Kim et al., 2025; Khoche et al., 2025; Lin et al., 2025b), the emerging 4D automotive radar is receiving increasing attention due to its lower cost, higher robustness under adverse weather conditions and the ability to measure extra radial relative velocity (RRV). However, since radar point clouds are significantly sparser and noisier than LiDAR, radar-based scene flow estimation is challenging and remains largely underexplored.

Due to the high cost of obtaining pointwise ground-truth 3D flow annotations for radar scene flow estimation, re-

cent studies have increasingly focused on *label-free* 4D radar scene flow estimation. A straightforward attempt is to extend LiDAR-based self-supervised approaches, but the commonly used clustering strategies (Zhang et al., 2024b; Lin et al., 2025b) and Chamfer-guided (Wu et al., 2020; Mittal et al., 2020; Pontes et al., 2020) pseudo correspondences are originally designed for high-performance LiDAR point clouds and cannot be easily applied to the challenging radar domain. To address the intractability of radar data, the pioneering RaFlow (Ding et al., 2022) introduces specialized self-supervised loss designs (Fig. 1a), using radar RRV as a supervision signal and relaxing LiDAR-oriented constraints to better adapt to radar conditions. However, the performance of RaFlow remains suboptimal due to the inherently low-fidelity nature of radar signals. Subsequent works (Ding et al., 2023; Zhai et al., 2025; Wu et al., 2025) adopt multi-task network architectures to leverage cross-modal supervision signals (Fig. 1b), leading to notable gains in flow prediction accuracy. Nevertheless, these frameworks are highly complicated and require seven or more loss terms for network training, including all self-supervised losses from RaFlow (Ding et al., 2022) as well as additional cross-modal losses derived from 3D LiDAR, 2D optical flow, and vehicle odometry. Notably, although these methods avoid using ground-truth scene flow labels, they instead generate pseudo radar flow labels from 3D tracking results on dense LiDAR point clouds.

Given the high cost of high-performance LiDAR sensors, we propose a novel setting, weakly supervised cross-modal learning for 4D radar scene flow, that relies *only on RGB images and odometry*, which are inexpensive sensors commonly available on consumer vehicles and robotic platforms. To the best of our knowledge, this is the *first* work to leverage RGB images and odometry exclusively as auxiliary supervision for 4D radar scene flow during training (Fig. 1c), while requiring only radar point clouds as input at inference time. Under this new setting, we design a task-specific lightweight framework, termed **IterFlow**. Compared to previous CMS methods, IterFlow is computational efficient and effective, which is trained with only three complementary losses, including two novel self-supervised losses guided by 2D semantics extracted from images together with a rigid static loss based on odometry for weak supervision.

IterFlow excels by incorporating *ball query-based cross-frame correlation* and an *iterative scene flow refinement* scheme. When calculating the correlation between frames, the ball query-based grouping is more robust than traditional K-Nearest Neighbor (KNN) grouping (Ward Jr, 1963; Ester et al., 1996) for radar point clouds, as it effectively avoids mismatches with distant points in sparse area. Moreover, iteratively updated scene flow enables IterFlow to refine coarse predicted flows from the initial steps and produce better scene flow estimation results than one-step inference

approaches (Cheng & Ko, 2022; Ding et al., 2022; 2023).

As for loss design, we first leverage instance-level semantic guidance from 2D images to establish two novel instance-aware losses. Specifically, 2D tracking and segmentation results are back-projected from the image plane into 3D space, assigning instance labels to valid radar points. Based on these labels, we define an *instance-aware Chamfer loss*, which computes the Chamfer distance only for point pairs belonging to the same instance across frames, and an *instance-level spatial smoothness loss*, which enforces flow coherence within each object. By integrating 2D semantic correspondence with 3D geometric relationships between frames, these instance-aware losses effectively mitigate the mismatches and ambiguities caused by commonly used all-pair Chamfer calculations and KNN-based flow smoothing, (Wu et al., 2020; Ding et al., 2022; 2023; Zhai et al., 2025; Wu et al., 2025) yielding semantically enriched supervision for radar scene flow learning. Additionally, we utilize vehicle odometry to construct a rigid static loss that supervises the flow estimation in static regions.

Our key contributions are summarized as follows:

- To the best of our knowledge, we are the first to propose a task-specific cross-modal framework for 4D radar scene flow estimation, using only images and odometry for weak supervision during training. IterFlow is lightweight, featuring iterative flow refinement scheme and ball query-based cross-frame correlation, both tailored to the challenging radar domain.
- We design two novel instance-aware losses by leveraging semantic guidance from images, effectively mitigating the mismatch and ambiguity issues commonly encountered in existing self-supervised losses.
- Our proposed method achieves state-of-the-art performance in 4D radar scene flow estimation without requiring flow labels, surpassing previous cross-modal supervised radar-based methods that require pseudo 3D flow labels from LiDAR data and even outperforming fully supervised models.

2. Related work

Supervised LiDAR Scene Flow. With advances on deep neural network on point sets (Qi et al., 2017a;b; Wu et al., 2019; Zhao et al., 2021; Wang et al., 2024; Sheng et al., 2025; Yuan et al., 2026), LiDAR-based supervised scene flow learning has been extensively studied (Liu et al., 2019; Puy et al., 2020; Wei et al., 2021; Cheng & Ko, 2022; Fu et al., 2023; Liu et al., 2024; Lin et al., 2025a) over the past few years. However, these methods focus on annotated synthetic datasets and confront the domain gap between real-world and synthetic data. To enable training on large-scale

automotive datasets which do not provide ready-made scene flow labels, a series of follow-up works (Jund et al., 2021; Zhang et al., 2024a; Kim et al., 2025; Khoche et al., 2025; Luo et al., 2025; Fu et al., 2026) utilize raw LiDAR point clouds and annotated tracking boxes to extract rigid flow labels for foreground objects. Using pillar-based (Lang et al., 2019) or voxel-based backbones (Tang et al., 2020), these methods can handle large-scale point clouds and have good real-time capability. Since LiDAR-based setups typically assume that ego motion is given, the performance of these methods is severely compromised when the vehicle’s odometry is unknown. Additionally, although recent works (Liu et al., 2022; Peng et al., 2023; Wan et al., 2023; Zhou et al., 2024) have incorporated 2D images or event cameras with LiDAR point clouds for multi-modal scene flow estimation, they directly use data from other modalities as input to the network.

Label-free LiDAR Scene Flow. To overcome the limitation of expensive 3D scene flow annotations, runtime optimization and self-supervised methods have witnessed remarkable progress in recent years. Although optimization-based methods (Li et al., 2021; 2023; Vedder et al., 2024) produce high quality flow predictions, they require excessive running time. In contrast, self-supervised deep learning methods enable faster inference and primarily rely on label-free proxy loss designs, including chamfer distance (Pontes et al., 2020), cycle-consistency (Mittal et al., 2020), local flow smoothness (Wu et al., 2020), and rigidity priors (Chodosh et al., 2024; Jiang et al., 2024; Wang et al., 2025). Some advanced researches further apply point grouping (Ward Jr, 1963; Ester et al., 1996) to cluster foreground points and construct self-supervised scene flow losses between point clusters (Zhang et al., 2024b; Lin & Caesar, 2024; Hoffmann et al., 2025; Lin et al., 2025b). However, LiDAR-based approaches naturally take advantage of rich spatial structural and geometric properties from high-resolution dense point clouds. Therefore, these methods cannot be easily applied to low-quality sparse radar point clouds.

Radar Scene Flow. Although remarkable progress has been made in LiDAR-based studies, research on challenging radar scene flow estimation is still in the starting stage. Due to laborious 3D point cloud annotation, existing radar-based solutions prefer settings free of scene flow labels. RaFlow (Ding et al., 2022) first employs radar RRV as a direct supervision for flow prediction and also introduces relaxed chamfer loss and spatial flow smoothness loss to cope with difficult radar data. However, the sparse and noisy nature of radar point clouds hinders the reliability of such exclusive self-supervised learning. CMFlow (Ding et al., 2023) then proposes a multi-task framework to leverage additional cross-modal supervision from dense LiDAR point clouds, optical flows and odometry. Recently, radar-based network design is becoming increasingly complex and loss

functions are constantly piling up to achieve higher performance. The latest (Zhai et al., 2025) adds decoupled feature extraction branches and flow refinement modules into previous network; Wu et al. (Wu et al., 2025) even integrates a pretrained 3D object detection network into the scene flow backbone. Unlike the mainstream trend, we strive to achieve effective radar scene flow estimation through lightweight task-specific network and concise loss design.

3. Method

Problem Definition. Radar scene flow estimation aims to recover the 3D motion field between two consecutive radar scans, $\mathbf{P}_t \in \mathbb{R}^{N_1 \times 5}$ and $\mathbf{P}_{t+1} \in \mathbb{R}^{N_2 \times 5}$, where N_1, N_2 denote the number of radar points in each frame. Every radar point consists of five attributes: its 3D coordinates, radar cross-section (RCS), and relative radial velocity (RRV). The coordinates of points in the source and target frame are denoted as $x_i \in \mathbf{P}_t$ and $y_i \in \mathbf{P}_{t+1}$, respectively. Following previous label-free settings (Ding et al., 2023; Wu et al., 2025; Zhai et al., 2025), while data from other modalities may be accessible during training, only radar data is required as input during inference. During testing, given $\mathbf{P}_t$ and $\mathbf{P}_{t+1}$, the model predicts the scene flow $\mathbf{F} \in \mathbb{R}^{N_1 \times 3}$ for each radar point in the source frame $\mathbf{P}_t$.

Method Overview. The overall architecture of our proposed method comprises an iterative scene flow estimation network, termed IterFlow (Fig. 2a), and a corresponding loss formulation component (Fig. 2b). During training, the consecutive radar point clouds $\mathbf{P}_t$ and $\mathbf{P}_{t+1}$ are fed into IterFlow to generate the final scene flow prediction $\mathbf{F}^K \in \mathbb{R}^{N_1 \times 3}$ after K steps of iterative refinement, which will be introduced in Sec. 3.1. Subsequently, auxiliary 2D image and odometry are used to construct three losses for optimizing the predicted flows: $\mathcal{L}_{total} = \mathcal{L}_{stat} + \mathcal{L}_{ic} + \mathcal{L}_{is}$. Here $\mathcal{L}_{stat}$ denotes the rigid static loss defined in Sec. 3.3; $\mathcal{L}_{ic}$ and $\mathcal{L}_{is}$ correspond to our instance-aware Chamfer loss and instance-level flow smoothness loss, respectively, which will be detailed in Sec. 3.2.

3.1. IterFlow

Previous radar-based scene flow estimation methods typically adopt KNN-based grouping for cross-frame correlation, such as patch-to-patch correlation in (Ding et al., 2022; 2023; Zhai et al., 2025) and cross-attention in (Wu et al., 2025). However, KNN grouping often leads to mismatches and ambiguities in sparse radar point clouds. Moreover, these approaches either produce final flow predictions in a single step or rely on additional auxiliary refinement modules. To address these limitations and achieve high-accuracy flow estimation on challenging 4D radar data, we propose IterFlow, a task-specific iterative network designed to refine scene flow estimation progressively (see Fig. 2a). Iter-

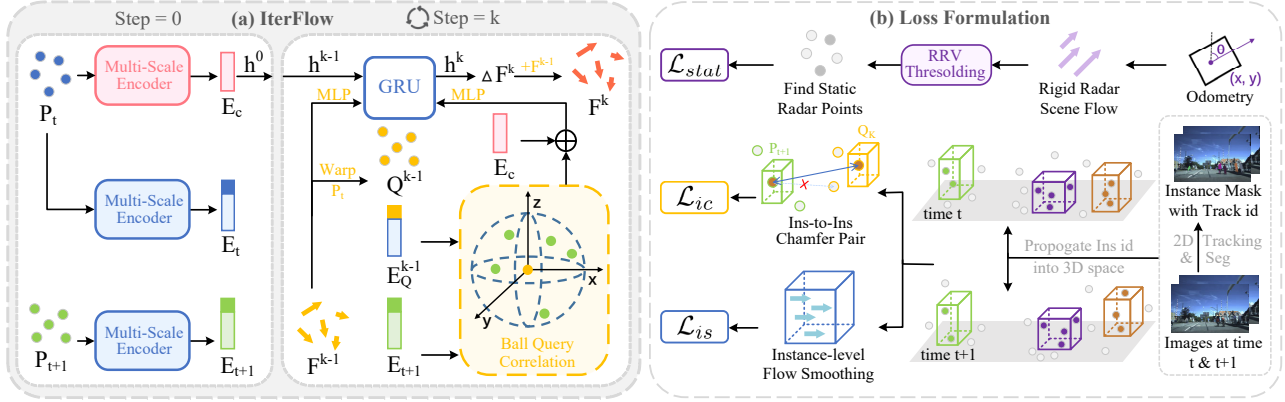


Figure 2. **Overall architecture of our proposed method.** The process of the k th scene flow iteration is depicted on the left and the detailed loss formulation process in the training stage is given on the right. $\oplus$ represents concatenation.

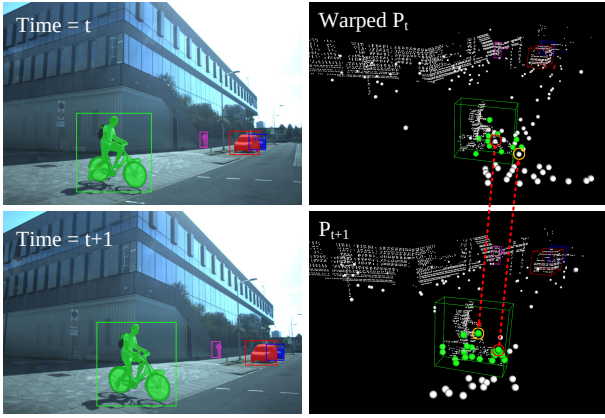


Figure 3. **An example of mismatched chamfer pairs.** The top and bottom rows show two consecutive frames at time t and time $t+1$, respectively. The smaller points are LiDAR points for auxiliary visualization, while the larger balls are radar points. P_t is first warped by estimated scene flow and then used to calculate chamfer loss with P_{t+1} . The orange circles indicate the current selected points for normal chamfer loss calculation, while the red circles and dashed lines highlight the possible mismatch between colored dynamic foreground points and white background points when instance-level guidance is unavailable.

Flow leverages ball query grouping to correlate cross-frame features within a constrained spatial range and employs a GRU-based (Cho et al., 2014) recurrent update scheme for iterative scene flow refinement.

In IterFlow, P_t and P_{t+1} share a multi-scale encoder (Ding et al., 2022; 2023) to extract point cloud features $E_t \in \mathbb{R}^{N_1 \times (3+C)}$ and $E_{t+1} \in \mathbb{R}^{N_2 \times (3+C)}$, respectively. Each pointwise feature $\varphi(x_i) \in E_t$ and $\varphi(y_i) \in E_{t+1}$ consists of the original input 3D position and the feature dimension C . Here $\varphi(\cdot)$ represents the multi-scale encoder. Following (Wei et al., 2021; Wang et al., 2023), another multi-scale encoder processes P_t to produce contextual features $E_c \in \mathbb{R}^{N_1 \times C}$, which initialize the hidden state of the GRU

unit for iterative residual flow generation.

During the k -th iteration, P_t is warped to Q^{k-1} using the previous flow estimation F^{k-1} , replacing the 3D coordinate part of E_t with the warped coordinates $q_i \in Q^{k-1}$ to form the warped features $\varphi(q_i) \in E_Q^{k-1}$. After that, a ball query operation with a limited search radius R is applied to each q_i to find its L nearest neighbors in P_{t+1} , denoted as $\mathcal{N}_L = \mathcal{N}_{P_{t+1}}(q_i)_L$. With set abstraction in (Qi et al., 2017a;b), the ball query-based cross-frame correlation feature is then computed as:

$$\mathbf{c}_i^k = \max_l (\text{MLP}(\text{concat}(\varphi(y_l) \cdot \varphi(q_i), y_l - q_i))). \quad (1)$$

The correlation feature $\mathbf{c}_i^k \in \mathbf{C}^k$ is concatenated with context feature E_c and fed into MLP, incorporating source-frame geometry to support flow prediction. Meanwhile, the coarse flow vector F_{k-1} is also encoded via an MLP. These features are fused to form the GRU input x_k , and the hidden state is updated as follows:

$$z^k = \sigma(\text{Conv}_{1d}([h^{k-1}, x^k], W_z)) \quad (2)$$

$$r^k = \sigma(\text{Conv}_{1d}([h^{k-1}, x^k], W_r)) \quad (3)$$

$$\hat{h}^k = \tanh(\text{Conv}_{1d}([r^k \odot h^{k-1}, x^k], W_h)) \quad (4)$$

$$h^k = (1 - z^k) \odot h^{k-1} + z^k \odot \hat{h}^k \quad (5)$$

where weight matrices W_z , W_r , and W_h can be learned during training. The updated h^k is fed into a convolutional flow head to produce residual flow ΔF^k , and the flow prediction is iteratively refined as: $F^k = F^{k-1} + \Delta F^k$.

3.2. Instance-aware Loss Functions

Pointwise Instance Label Generation¹. As depicted in Fig. 2b, the images at time t and $t+1$ are fed into

¹Please refer to Sec. B.1 in the appendix for detailed process of generating instance label for each radar point.

off-the-shelf 2D tracking (Khanam & Hussain, 2024) and pretrained segmentation models (Kirillov et al., 2023) to generate instance segmentation masks with tracked IDs. Through 2D-to-3D back-projection, corresponding 2D instance label $g \in 1, 2, 3, \dots, G$ or background label $g = 0$ can be propagated into 3D space and assigned to each 3D radar point. Here, G denotes the maximum number of foreground instances observed across the source and target frames. The retrieved pointwise 3D instance label is denoted as $\phi(\cdot)$. The radar points are then grouped by their instance labels to form instance-level point sets in both frames: $\mathbf{S}_t^g = \{x_i | \phi(x_i) = g, x_i \in \mathbf{P}_t\}$ and $\mathbf{S}_{t+1}^g = \{y_j | \phi(y_j) = g, y_j \in \mathbf{P}_{t+1}\}$.

Instance-aware Chamfer Loss. The commonly used Chamfer loss is calculated based on the mutual nearest neighbor distance between two point clouds. However, due to the inherent sparsity and disorder of point clouds, mismatched Chamfer pairs are inevitable, and this issue becomes more pronounced with sparse 4D radar data. As illustrated in Fig. 3, radar points on objects are sparse and often surrounded by static background points or noise. In such cases, the nearest neighbor of a dynamic foreground point in $\mathbf{P}_{t+1}$ within the warped $\mathbf{P}_t$ may incorrectly correspond to a background point, and vice versa. This misalignment significantly hinders the network’s ability to learn accurate motion for dynamic foreground objects.

To mitigate this issue, we propose calculating the Chamfer loss exclusively between point pairs that belong to the same instance, utilizing the pointwise instance label obtained in the previous step (e.g. green cyclist points in warped $\mathbf{P}_t$ and $\mathbf{P}_{t+1}$). With assistance from 2D semantic information, our instance-aware Chamfer loss $\mathcal{L}_{ic}$ (Eq. 6) effectively reduces the negative impact of mismatched point pairs and prevents confusion between dynamic foreground and static background points.

$$\mathcal{L}_{ic} = \frac{1}{G} \sum_{g=1}^G \left(\sum_{x_i \in \mathbf{S}_t^g} \frac{D(\hat{\mathbf{P}}_{t,i}, \mathbf{P}_{t+1})}{|\mathbf{S}_t^g|} + \sum_{y_j \in \mathbf{S}_{t+1}^g} \frac{D(\mathbf{P}_{t+1,j}, \hat{\mathbf{P}}_t)}{|\mathbf{S}_{t+1}^g|} \right), \quad (6)$$

where $\hat{\mathbf{P}}_t$ is the source frame point cloud $\mathbf{P}_t$ warped by the predicted scene flow, and $D(p, \mathbf{P})$ is the minimum distance between point q and its nearest neighbor in point cloud $\mathbf{P}$:

$$D(q, \mathbf{P}) = \min_{p_m \in \mathbf{P}} \|q - p_m\|_2^2 \quad (7)$$

Instance-level Flow Smoothness Loss. Previous works (Ding et al., 2022; 2023; Zhai et al., 2025; Wu et al., 2025) typically adopt a spatial smoothness loss that enforces flow consistency within a KNN neighborhood. However, this KNN-based flow smoothing introduces potential risks of incorrect flow regularization between dynamic foreground and static background points, especially since radar point clouds are unevenly distributed and often extremely sparse in certain regions. A real-world example is shown in Fig. 4,

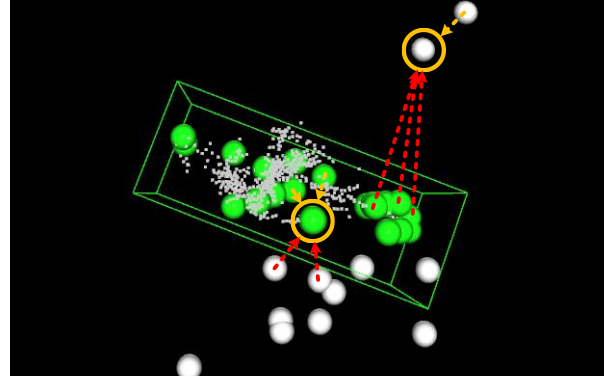


Figure 4. An example of wrong KNN-based spatial flow smoothing. The figure shows the bird’s-eye view of a cyclist (green bounding box). Foreground radar points of the cyclist are painted in green. The orange circles highlight the current center points for KNN search, while the red dashed lines represent the confusion between foreground dynamic and background static points caused by the KNN-based flow smoothing in previous methods.

where both foreground and background points are prone to mismatches during KNN-based flow smoothing. The resulting enforced consistency between incorrect point pairs can significantly degrade network performance.

To address this problem, we introduce an instance-level flow smoothness loss $\mathcal{L}_{is}$. Unlike previous methods that apply flow consistency constraints for all radar points in the source frame, $\mathcal{L}_{is}$ focuses exclusively on maintaining flow smoothness within each foreground instance. This design effectively mitigates the influence of noisy background points and prevents mismatches between dynamic foreground and static regions. The mean predicted scene flow of each instance is calculated as: $\mathbf{F}_{mean}^g = \frac{1}{|\mathbf{S}_t^g|} \sum_{x_i \in \mathbf{S}_t^g} \mathbf{F}_i^K$, and $\mathcal{L}_{is}$ is formulated as:

$$\mathcal{L}_{is} = \frac{1}{G} \sum_{g=1}^G \frac{1}{|\mathbf{S}_t^g|} \sum_{x_i \in \mathbf{S}_t^g} \|\mathbf{F}_i^K - \mathbf{F}_{mean}^g\|_2. \quad (8)$$

3.3. Rigid Static Loss

The rigid static loss $\mathcal{L}_{stat}$ measures the difference between the estimated scene flow of static points and their corresponding rigid flow based on the ground-truth vehicle odometry. Following previous methods (Wu et al., 2025; Zhai et al., 2025), we first compensate radar RRV with known ego motion to obtain absolute radial velocity (ARV) for each point in $\mathbf{P}_t$. After that, static radar points can be identified by ARV thresholding and merged into a static point set $\mathbf{S}_r = \{x_i | ARV_i < \gamma_{thre}, x_i \in \mathbf{P}_t\}$. Then $\mathcal{L}_{stat}$ can be formulated as Eq. 9, where $\mathbf{T}$ is the ground-truth ego-motion transformation.

$$\mathcal{L}_{stat} = \frac{1}{|\mathbf{S}_r|} \sum_{x_i \in \mathbf{S}_r} \|(\mathbf{F}_i^K - (\mathbf{T} \circ x_i - x_i))\|_2, \quad (9)$$

Table 1. Quantitative Evaluation and Model Complexity Comparison on VoD validation set. In the Category (Cat.) column, existing methods are classified depending on the input modality used in their original work. In the Supervision (Sup.) column, the methods are categorized as fully supervised (Full), self-supervised (Self), or cross-supervised (Cross). R represents radar point clouds input.

Cat.	Method	Inp.	Sup.	Overall				Moving	Static	Model Complexity	
				EPE [m]↓	AccS [%]↑	AccR [%]↑	RNE [m]↓	MRNE [m]↓	SRNE [m]↓	Params [B]↓	GFLOPs↓
LiDAR-based	Flow4D (Kim et al., 2025)	R	Full	0.2555	4.48	19.88	0.1032	0.1194	0.1007	4.6M	-
	BiFlow (Cheng & Ko, 2022)	R	Full	0.2140	18.14	35.07	0.0859	0.1235	0.0823	8.2M	1.01
	DeFlow (Zhang et al., 2024a)	R	Full	0.2015	8.94	29.01	0.0813	0.0981	0.0789	6.9M	-
	PointPWC (Wu et al., 2020)	R	Full	0.1271	23.80	53.00	0.0512	0.0859	0.0478	7.7M	1.26
	PV-RAFT (Wei et al., 2021)	R	Full	0.1140	23.24	58.07	0.0459	0.0861	0.0418	192K	0.63
	PointPWC (Wu et al., 2020)	R	Self	0.4314	0.02	0.17	0.1735	0.1703	0.1731	7.7M	1.26
	FlowStep3D (Kittenplon et al., 2021)	R	Self	0.2607	2.54	14.62	0.1050	0.1261	0.1025	689K	0.85
Radar-based	RaFlow (Ding et al., 2022)	R	Self	0.2753	3.49	17.48	0.1105	0.1372	0.1081	4.1M	12.78
	CMFlow (Ding et al., 2023)	R	Cross	0.1600	19.83	42.38	0.0643	0.0838	0.0625	4.2M	12.83
	IterFlow (ours)	R	Cross	0.1045	33.40	63.75	0.0420	0.0833	0.0380	113K	0.40

Table 2. Quantitative Evaluation on Network Architecture and Loss Scalability on VoD validation set. $\mathcal{L}_c$, $\mathcal{L}_s$, $\mathcal{L}_g$ denote the official chamfer loss, local flow smoothness loss and local geometric consistency loss from PointPWC (Wu et al., 2020). $\mathcal{L}_{rd}$, $\mathcal{L}_{sc}$, $\mathcal{L}_{ss}$ represent the radial displacement loss, soft chamfer loss and spatial smoothness loss from RaFlow (Ding et al., 2022).

Method	Loss	Overall				Moving	Static
		EPE [m]↓	AccS [%]↑	AccR [%]↑	RNE [m]↓	MRNE [m]↓	SRNE [m]↓
1) PointPWC (Wu et al., 2020)	(Wu et al., 2020): $\mathcal{L}_c + \mathcal{L}_s + \mathcal{L}_g$	0.4314	0.02	0.17	0.1735	0.1703	0.1731
2) PointPWC (Wu et al., 2020)	Ours: $\mathcal{L}_{stat} + \mathcal{L}_{ic} + \mathcal{L}_{is}$	0.1228	25.07	54.45	0.0495	0.0875	0.0457
3) RaFlow (Ding et al., 2022)	(Ding et al., 2022): $\mathcal{L}_{rd} + \mathcal{L}_{sc} + \mathcal{L}_{ss}$	0.2753	3.49	17.48	0.1105	0.1372	0.1081
4) RaFlow (Ding et al., 2022)	Ours: $\mathcal{L}_{stat} + \mathcal{L}_{ic} + \mathcal{L}_{is}$	0.1163	28.90	58.30	0.0468	0.0917	0.0426
5) IterFlow	(Wu et al., 2020): $\mathcal{L}_c + \mathcal{L}_s + \mathcal{L}_g$	0.4084	2.13	12.05	0.1638	0.1280	0.1669
6) IterFlow	(Ding et al., 2022): $\mathcal{L}_{rd} + \mathcal{L}_{sc} + \mathcal{L}_{ss}$	0.1872	19.53	41.44	0.0751	0.0851	0.0740
7) IterFlow	Ours: $\mathcal{L}_{stat} + \mathcal{L}_{ic} + \mathcal{L}_{is}$	0.1045	33.40	63.75	0.0420	0.0833	0.0380

4. Experiments

Datasets. Following (Ding et al., 2023; Zhai et al., 2025; Wu et al., 2025), we conduct experiments on the real-world View-of-Delft (VoD) dataset (Palffy et al., 2022), which provides 8,600 frames of synchronized and calibrated odometry, 3D LiDAR point clouds, 4D radar data and RGB images. The average ratio between radar and LiDAR resolution is 2.5 on the VoD dataset. The radar points within the viewing frustum are taken as input, which are restricted in a height range of $[-3\text{m}, 3\text{m}]$. Since the VoD dataset does not provide ready-made scene flow ground truth, we adopt the commonly used preprocessing methods to generate scene flow labels from annotated 3D tracking boxes for the training set (5,066 frames) and validation set (1,273 frames) according to the official split².

Implementation Details. Following the previous radar-based methods (Ding et al., 2022; 2023), $N_1 = N_2 = 256$ radar points are randomly sampled in each frame as input and the ARV threshold γ_{thre} is set to 0.1m/s to identify static points. For the ball query correlation, we sample $L = 8$ neighbors with a limited search radius of $R = 1\text{m}$. Scene flow is updated for $K = 12$ iterations in both training and evaluation stage³. In order to generate reliable instance labels for radar points from 2D images, we adopt the

officially released YOLO11-l (Khanam & Hussain, 2024) model and the huge version of Segment Anything Model (SAM) (Kirillov et al., 2023) for 2D tracking and segmentation. IterFlow is implemented in PyTorch (Paszke et al., 2019) and trained for 150 epochs with a batch size of 8. The Adam optimizer with an initial learning rate of 0.001 is used. For a fair comparison with the baselines, we use their official loss configuration and hyperparameter settings for network retraining on the VoD radar scene flow dataset. In addition, for the cross-modal supervised CMFlow (Ding et al., 2023), we generate extra required optical flow labels and pseudo scene flow labels by adopting their officially released pretrained 2D optical flow estimation model and 3D tracking results.

Evaluation Metrics. Following (Ding et al., 2022; 2023; Zhai et al., 2025; Wu et al., 2025), we use standard metrics for evaluation: 1) EPE: average 3D end-point-error measures the L2 distance between ground truth flow vectors and scene flow predictions; 2) AccS/AccR: the ratio of points that meet a strict/relaxed condition, *i.e.* $EPE < 0.05/0.1\text{m}$ or the relative error $< 5\%/10\%$; 3) RNE: resolution-normalized EPE by dividing EPE by the ratio of 4D radar and LiDAR resolution, to accommodate radar sensors with different resolution. 4) MRNE/SRNE: RNE for moving points and static points, respectively.

4.1. Main Results

Overall Quantitative Comparison. We compare our method with state-of-the-art scene flow estimation models

²Please refer to Sec. A.1 and Sec. A.2 in appendix for detailed introduction of VoD dataset and process of annotating rigid scene flow based on 3D tracking boxes.

³Please refer to Sec. B.2 in the appendix for hyperparameter sensitivity analysis of L , R and K .

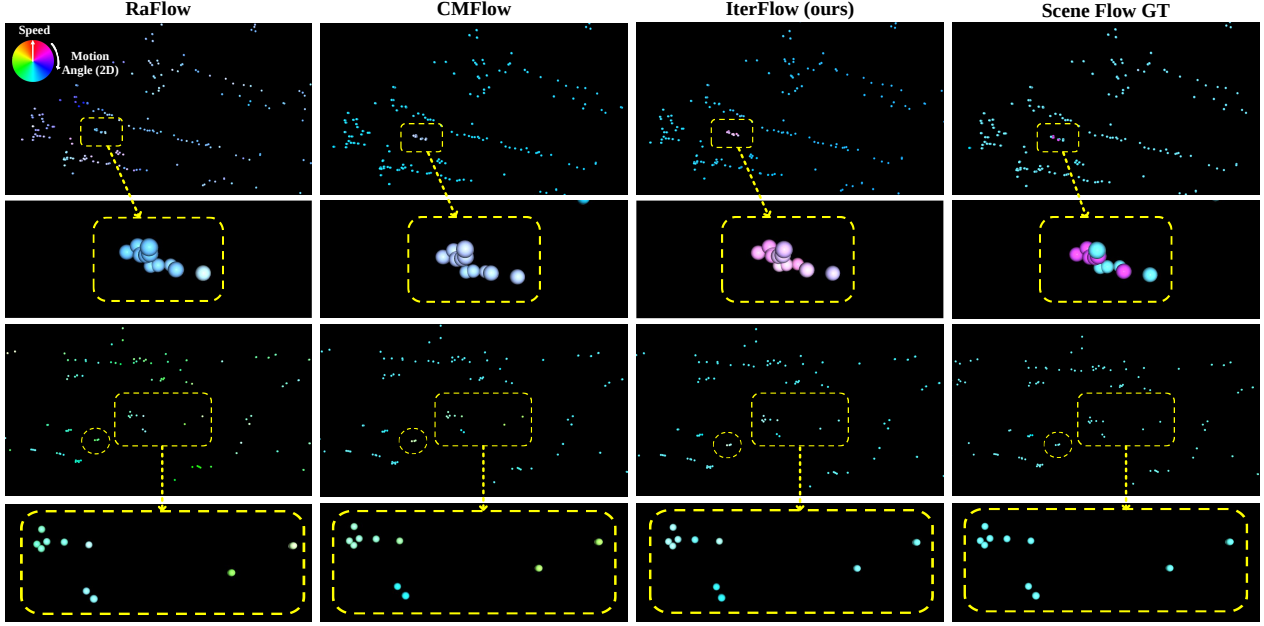


Figure 5. **Qualitative Results on VoD validation dataset.** The first row and the third row display two separate traffic scenes; while the second row and fourth row show the zoomed-in results of the regions highlighted with yellow rectangles. As shown in the legend, the direction and magnitude of scene flow vectors are employed as hue and saturation, respectively.

on the VoD dataset and the experimental results are summarized in Table 1. Note that fully-supervised methods are trained with the radar scene flow ground truth derived from the annotated 3D tracking boxes provided by the dataset, and CMFlow (Ding et al., 2023) is trained with cross-modal supervisions including pseudo scene flow generated based on the 3D tracking results from a pretrained multi-object tracking model (Weng et al., 2020); both are supervised by direct 3D motion cues, while the remaining methods (including ours) operate without access to ground-truth tracking boxes or dense LiDAR point clouds.

Compared to the previous best radar-based model, CMFlow (Ding et al., 2023), which employs a complex multi-task framework and uses a total of seven loss terms for training (including three self-supervised losses and four cross-modal losses), our task-specific IterFlow is trained with only three losses and achieves superior performance across all evaluation metrics. In particular, IterFlow yields a 34.7% performance improvement on the EPE metric, while increasing AccS and AccR by 13.6% and 21.4%, respectively. IterFlow also brings a 39.2% error reduction on the SRNE metric and achieves the best performance on the MRNE metric, generating more accurate flow estimation in static regions while maintaining strong performance in dynamic areas. This demonstrates that effectively leveraging instance-aware guidance from 2D images allows our weakly supervised cross-modal learning framework to substantially outperform CMFlow, which relies on pseudo 3D scene flow and 2D optical flow labels for supervision.

When compared to LiDAR-based approaches, our method even surpasses the best fully supervised model, PV-RAFT (Wei et al., 2021), achieving performance improvements across all metrics, enhancing the prediction accuracy on the EPE metric by 8.3%. This result highlights that our ball query-based correlation operation is more robust in sparse radar scenarios than the KNN-based and voxel-based correlation modules used in PV-RAFT. Overall, our approach establishes a new state of the art on the VoD radar scene flow dataset, validating the effectiveness of our network architecture and loss design.

Model Size and Computational Cost Comparison. As shown in the rightmost two columns in Table 1, our proposed IterFlow is computational efficient and requires the minimum parameter size among all scene flow estimation methods. Compared to the best radar-based CMFlow (Ding et al., 2023), IterFlow has $\sim 40\times$ fewer parameters and $\sim 30\times$ lower GFLOPs while achieving better performance. It illustrates that our task-specific network design is more lightweight and effective. In particular, PV-RAFT (Wei et al., 2021) and FlowStep3D (Kittenplon et al., 2021) are also iterative, but these methods require more parameters than ours. This is because IterFlow adopts a more concise yet effective ball query-based cross-frame correlation design, rather than the original KNN-based and multi-scale voxel-based correlation branches in PV-RAFT and global correlation in FlowStep3D.

Experiments on IterFlow and Loss Scalability. To fur-

Table 3. Ablation Study on Loss Terms on VoD validation set. $\mathcal{L}_{sc}$ is the soft chamfer loss without instance-aware guidance and $\mathcal{L}_{ss}$ is the KNN-based spatial smoothness loss; both are from RaFlow (Ding et al., 2022). * denotes using the pointwise instance labels from annotated 3D tracking boxes in VoD dataset for instance-aware loss calculation.

Loss	Overall				Moving	Static
	EPE [m]↓	AccS [%]↑	AccR [%]↑	RNE [m]↓	MRNE [m]↓	SRNE [m]↓
$\mathcal{L}_{stat} + \mathcal{L}_{sc}$	0.1512	23.47	51.11	0.0609	0.1021	0.0571
$\mathcal{L}_{stat} + \mathcal{L}_{ic}$	0.1249	28.83	58.28	0.0503	0.0937	0.0461
$\mathcal{L}_{stat} + \mathcal{L}_{ic} + \mathcal{L}_{ss}$	0.1191	28.76	57.43	0.0479	0.0778	0.0447
$\mathcal{L}_{stat} + \mathcal{L}_{ic} + \mathcal{L}_{is}$	0.1045	33.40	63.75	0.0420	0.0833	0.0380
$\mathcal{L}_{stat} + \mathcal{L}_{ic}^* + \mathcal{L}_{is}^*$	0.1041	34.12	64.42	0.0419	0.0740	0.0385

Table 4. Ablation Study on Grouping Method for Cross-frame Correlation on VoD validation set.

Grouping Method	Overall				Moving	Static
	EPE [m]↓	AccS [%]↑	AccR [%]↑	RNE [m]↓	MRNE [m]↓	SRNE [m]↓
KNN	0.1817	19.68	40.98	0.0729	0.1028	0.0703
Ball Query	0.1045	33.40	63.75	0.0420	0.0833	0.0380

ther validate the effectiveness of our IterFlow network and loss designs, we conduct cross-comparative experiments between different architectures and loss combinations. Specifically, we adopt self-supervised loss functions from LiDAR-based PointPWC (Wu et al., 2020) and radar-based RaFlow (Ding et al., 2022) to train IterFlow, and conversely apply our proposed loss functions to train the original PointPWC and RaFlow networks.

As shown in the last section of Table 2, when replacing the backbone of PointPWC and RaFlow with IterFlow while keeping their respective loss combinations (rows 5 and 6), the models exhibit consistently superior performance compared to the original versions (rows 1 and 3). Notably, when substituting the single-inference RaFlow (row 3) with our iterative IterFlow architecture (row 6), we observe a substantial improvement, with an EPE reduction of 33.6%, demonstrating the clear advantage of our network design.

Furthermore, when replacing the original loss combinations in (Wu et al., 2020) and (Ding et al., 2022) with our proposed ones (rows 2 and 4), both networks achieve notable performance gains across all evaluation metrics. In particular, the EPE decreases by 71.5% for PointPWC (row 2 vs row 1) and 57.8% for RaFlow (row 4 vs row 3), showing that our loss functions can effectively generalize to other architectures and substantially enhance scene flow prediction accuracy.

Finally, comparing rows 5 and 7, we find that using LiDAR-based losses in (Wu et al., 2020) to train our IterFlow results in dramatic performance degradation on all evaluation metrics, highlighting that generic LiDAR-oriented self-supervised losses do not work well for radar-based scenarios. This analysis underscores the necessity and effectiveness of our loss designs for radar scene flow learning.

Qualitative Results. In Fig. 5, we compare the qualitative results of our IterFlow with the state-of-the-art radar-based scene flow estimation methods, with the ground truth scene flow serving as a reference. In general, our method performs much better than RaFlow (Ding et al., 2022), generating reliable scene flow predictions that best match the ground truth. Although CMFlow (Ding et al., 2023) has also demonstrated good performance, our IterFlow performs better in some difficult sparse areas, as highlighted with yellow rectangles.

4.2. Ablation Studies

Ablation Study on Loss Terms. Various ablation studies are conducted on IterFlow to investigate the advantage of each loss term. First, while keeping the rigid static loss $\mathcal{L}_{stat}$ unchanged, we replace our instance-aware chamfer loss $\mathcal{L}_{ic}$ with the soft chamfer loss $\mathcal{L}_{sc}$ from RaFlow (Ding et al., 2022) to train the network. As shown in Table 3, all evaluation metrics decrease after replacing $\mathcal{L}_{ic}$, with overall performance on the EPE metric compromised by 21.1%. The advantage of $\mathcal{L}_{ic}$ over $\mathcal{L}_{sc}$ is twofold: on one hand, $\mathcal{L}_{ic}$ only calculates the chamfer distance between points within the same instance across frames, which effectively reduces confusion caused by mismatched point pairs; on the other hand, $\mathcal{L}_{ic}$ is more robust to noise because the chamfer loss of points that cannot be successfully matched with any instance is not included in $\mathcal{L}_{ic}$.

Second, we examine the effectiveness of $\mathcal{L}_{is}$ by removing it from total loss. The experimental results in Table 3 illustrate that the addition of $\mathcal{L}_{is}$ successfully improves the prediction accuracy in both dynamic and static areas in the scene, achieving a performance improvement of 16.3% on the overall EPE metric. As shown in Table 3, directly replacing $\mathcal{L}_{ss}$ with $\mathcal{L}_{is}$ leads to a slight decrease in performance on moving objects, because dynamic foreground points only occupy

a very small fraction of the scene and are more susceptible to 2D instance-level semantic accuracy. Moreover, it can be observed that with instance labels from 3D tracking ground truth, $\mathcal{L}_{ic}^*$ and $\mathcal{L}_{is}^*$ effectively promote the scene flow prediction performance in the moving areas. In general, our $\mathcal{L}_{is}$ is more robust than KNN-based $\mathcal{L}_{ss}$, because $\mathcal{L}_{is}$ naturally avoids forced flow smoothing between dynamic foreground and static background points in sparse radar point clouds. This analysis demonstrates that establishing image-guided instance-level constraints promotes a more comprehensive understanding of the dynamic environment and powerfully contributes to scene flow prediction.

Ablation Study on Grouping Method. To verify the advantage of ball query-based grouping over commonly used KNN grouping for cross-frame correlation in sparse radar scenes, we replace the original ball query-based grouping in IterFlow with naive KNN grouping. As shown in Table 4, ball query-based grouping proves to be more robust in sparse radar point clouds, which consistently outperforms KNN grouping and improves the network performance on overall EPE metric by 42.5%. This is because KNN always returns exactly L neighbors based on distance rank, even when some of them are still spatially far away. In contrast, ball query first enforces spatial locality and then samples up to L neighbors within a radius R , so different groups may contain different numbers of points. In sparse radar regions, this prevents cross-frame correspondences between points that are close only in rank but not in absolute distance, thereby reducing mismatches and improving robustness.

Dependence Analysis of 2D Tracking and Segmentation

Generating pointwise instance labels through off-the-shelf camera-based models is an important component to our weakly-supervised cross-modal framework. As shown in Table 3, IterFlow trained with $\mathcal{L}_{ic}$ and $\mathcal{L}_{is}$ has already performed nearly as $\mathcal{L}_{ic}^*$ and $\mathcal{L}_{is}^*$ on overall metrics. This demonstrates that, by effectively leveraging semantic information from images, our proposed image-based instance-aware loss functions can promisingly substitute for the requirement of 3D tracking labels. More experimental results with various combinations of 2D pipelines are given in Sec. C.4 in the appendix.

5. Conclusion

In this paper, we address 4D radar scene flow estimation through a new setting, where only RGB images and odometry available for auxiliary supervision during training. We introduce a novel framework that breaks from the prevailing trend of increasing network and loss complexity in existing methods. Specifically, we propose IterFlow, a lightweight yet effective network trained by only three targeted losses, including two instance-aware terms guided by 2D tracking

and segmentation results, and one rigid static term based on odometry. Without any 3D motion supervision, our method surpasses cross-modal supervised and even fully supervised models, highlighting the potential of leveraging 2D semantic guidance for 4D radar scene flow estimation.

Acknowledgements

This work was conducted during Jingyun’s visit to the IMPL Lab at SUTD, funded by China Scholarship Council. This research was supported by the Ministry of Education, Singapore, under its MOE Academic Research Fund Tier 2 (MOE-T2EP20124-0013). This work was also supported by the Key Project of Natural Science Foundation of Zhejiang Province under Grant LZ26F010003, the Key Research & Development Plan of Zhejiang Province under Grant No.2024C01010, 2024C01017, 2025C01039, the Joint R&D Program of the Yangtze River Delta Community of Sci-Tech Innovation with grant number 2024CSJGG01000.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Cheng, W. and Ko, J. H. Bi-pointflownet: Bidirectional learning for point cloud based scene flow estimation. In *European Conference on Computer Vision*, pp. 108–124. Springer Nature Switzerland, 2022.
- Cho, K., Van Merriënboer, B., Bahdanau, D., and Bengio, Y. On the properties of neural machine translation: Encoder-decoder approaches. *arXiv preprint arXiv:1409.1259*, 2014.
- Chodosh, N., Ramanan, D., and Lucey, S. Re-evaluating lidar scene flow. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 6005–6015, 2024.
- Ding, F., Pan, Z., Deng, Y., Deng, J., and Lu, C. X. Self-supervised scene flow estimation with 4-d automotive radar. *IEEE Robotics and Automation Letters*, 7(3):8233–8240, 2022.
- Ding, F., Palffy, A., Gavrila, D. M., and Lu, C. X. Hidden gems: 4d radar scene flow learning using cross-modal supervision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9340–9349, 2023.

- Ester, M., Kriegel, H.-P., Sander, J., Xu, X., et al. A density-based algorithm for discovering clusters in large spatial databases with noise. In *kdd*, volume 96, pp. 226–231, 1996.
- Fu, J., Xiang, Z., Qiao, C., and Bai, T. Pt-flownet: Scene flow estimation on point clouds with point transformer. *IEEE Robotics and Automation Letters*, 8(5):2566–2573, 2023.
- Fu, J., Xiang, Z., and Zhao, N. Raliflow: Scene flow estimation with 4d radar and lidar point clouds. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 4012–4021, 2026.
- Gallagher, J. E. and Oughton, E. J. Surveying you only look once (yolo) multispectral object detection advancements, applications and challenges. *IEEE Access*, 2025.
- Hoffmann, D. T., Raza, S. H., Jiang, H., Tananaev, D., Klingenhoefer, S., and Meinke, M. Floxels: Fast unsupervised voxel based scene flow estimation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 22328–22337, 2025.
- Jiang, C., Wang, G., Liu, J., Wang, H., Ma, Z., Liu, Z., Liang, Z., Shan, Y., and Du, D. 3dsflabelling: Boosting 3d scene flow estimation by pseudo auto-labelling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15173–15183, 2024.
- Jund, P., Sweeney, C., Abdo, N., Chen, Z., and Shlens, J. Scalable scene flow from point clouds in the real world. *IEEE Robotics and Automation Letters*, 7(2):1589–1596, 2021.
- Khanam, R. and Hussain, M. Yolov11: An overview of the key architectural enhancements. *arXiv preprint arXiv:2410.17725*, 2024.
- Khatri, I., Vedder, K., Peri, N., Ramanan, D., and Hays, J. I can’t believe it’s not scene flow! In *European Conference on Computer Vision*, pp. 242–257. Springer, 2024.
- Khoche, A., Zhang, Q., Sanchez, L. P., Asefaw, A., Mansouri, S. S., and Jensfelt, P. Ssf: Sparse long-range scene flow for autonomous driving. *arXiv preprint arXiv:2501.17821*, 2025.
- Kim, J., Woo, J., Shin, U., Oh, J., and Im, S. Flow4d: Leveraging 4d voxel network for lidar scene flow estimation. *IEEE Robotics and Automation Letters*, 2025.
- Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W.-Y., et al. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 4015–4026, 2023.
- Kittenplon, Y., Eldar, Y. C., and Raviv, D. Flowstep3d: Model unrolling for self-supervised scene flow estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4114–4123, 2021.
- Lang, A. H., Vora, S., Caesar, H., Zhou, L., Yang, J., and Beijbom, O. Pointpillars: Fast encoders for object detection from point clouds. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 12697–12705, 2019.
- Li, X., Kaesemodel Pontes, J., and Lucey, S. Neural scene flow prior. *Advances in Neural Information Processing Systems*, 34:7838–7851, 2021.
- Li, X., Zheng, J., Ferroni, F., Pontes, J. K., and Lucey, S. Fast neural scene flow. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9878–9890, 2023.
- Lin, M., Xu, G., Wang, Y., Wang, X., and Yang, X. Flowmamba: Learning point cloud scene flow with global motion propagation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 5271–5279, 2025a.
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., and Zitnick, C. L. Microsoft coco: Common objects in context. In *European conference on computer vision*, pp. 740–755. Springer, 2014.
- Lin, Y. and Caesar, H. Icp-flow: Lidar scene flow estimation with icp. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15501–15511, 2024.
- Lin, Y., Wang, S., Nan, L., Kooij, J., and Caesar, H. Vote-flow: Enforcing local rigidity in self-supervised scene flow. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 17155–17164, 2025b.
- Liu, H., Lu, T., Xu, Y., Liu, J., Li, W., and Chen, L. Camliflow: bidirectional camera-lidar fusion for joint optical flow and scene flow estimation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 5791–5801, 2022.
- Liu, J., Wang, G., Ye, W., Jiang, C., Han, J., Liu, Z., Zhang, G., Du, D., and Wang, H. Diffflow3d: Toward robust uncertainty-aware scene flow estimation with iterative diffusion-based refinement. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15109–15119, 2024.
- Liu, X., Qi, C. R., and Guibas, L. J. FlowNet3D: Learning scene flow in 3D point clouds. In *CVPR*, pp. 529–537, 2019.

- Luo, J., Cheng, J., Tang, X., Zhang, Q., Xue, B., and Fan, R. Mambaflow: A novel and flow-guided state space model for scene flow estimation. *arXiv preprint arXiv:2502.16907*, 2025.
- Mittal, H., Okorn, B., and Held, D. Just go with the flow: Self-supervised scene flow estimation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 11177–11185, 2020.
- Palfy, A., Pool, E., Baratam, S., Kooij, J. F., and Gavrila, D. M. Multi-class Road User Detection with 3+ 1D Radar in the View-of-Delft Dataset. *RA-L*, 7(2):4961–4968, 2022.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- Peng, C., Wang, G., Lo, X. W., Wu, X., Xu, C., Tomizuka, M., Zhan, W., and Wang, H. Delflow: Dense efficient learning of scene flow for large-scale point clouds. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 16901–16910, 2023.
- Pontes, J. K., Hays, J., and Lucey, S. Scene flow from point clouds with or without learning. In *2020 international conference on 3D vision (3DV)*, pp. 261–270. IEEE, 2020.
- Puy, G., Boulch, A., and Marlet, R. Flot: Scene flow on point clouds guided by optimal transport. In *European conference on computer vision*, pp. 527–544. Springer, 2020.
- Qi, C. R., Su, H., Mo, K., and Guibas, L. J. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *CVPR*, pp. 652–660, 2017a.
- Qi, C. R., Yi, L., Su, H., and Guibas, L. J. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. *NIPS*, 30, 2017b.
- Ravi, N., Gabeur, V., Hu, Y.-T., Hu, R., Ryali, C., Ma, T., Khedr, H., Rädle, R., Rolland, C., Gustafson, L., et al. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024.
- Sheng, H., Cai, S., Zhao, N., Deng, B., Liang, Q., Zhao, M.-J., and Ye, J. Ct3d++: Improving 3d object detection with keypoint-induced channel-wise transformer. *International Journal of Computer Vision*, 133(7):4817–4836, 2025.
- Tang, H., Liu, Z., Zhao, S., Lin, Y., Lin, J., Wang, H., and Han, S. Searching efficient 3d architectures with sparse point-voxel convolution. In *European conference on computer vision*, pp. 685–702. Springer, 2020.
- Vedder, K., Peri, N., Khatri, I., Li, S., Eaton, E., Kocamaz, M., Wang, Y., Yu, Z., Ramanan, D., and Pehserl, J. Neural eulerian scene flow fields. *arXiv preprint arXiv:2410.02031*, 2024.
- Wan, Z., Mao, Y., Zhang, J., and Dai, Y. Rpeflow: Multimodal fusion of rgb-pointcloud-event for joint optical flow and scene flow estimation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 10030–10040, 2023.
- Wang, G., Feng, Z., Jiang, C., Liu, J., and Wang, H. Unsupervised learning of 3d scene flow with lidar odometry assistance. *IEEE Transactions on Intelligent Transportation Systems*, 2025.
- Wang, J., Cheng, Z., Zhao, N., Cheng, J., and Yang, X. On-the-fly point feature representation for point clouds analysis. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pp. 9204–9213, 2024.
- Wang, Z., Wei, Y., Rao, Y., Zhou, J., and Lu, J. 3d point-voxel correlation fields for scene flow estimation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(11):13621–13635, 2023.
- Ward Jr, J. H. Hierarchical grouping to optimize an objective function. *Journal of the American statistical association*, 58(301):236–244, 1963.
- Wei, Y., Wang, Z., Rao, Y., Lu, J., and Zhou, J. Pv-raft: Point-voxel correlation fields for scene flow estimation of point clouds. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 6954–6963, 2021.
- Weng, X., Wang, J., Held, D., and Kitani, K. 3d multi-object tracking: A baseline and new evaluation metrics. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 10359–10366. IEEE, 2020.
- Wu, J., Braun, M., Spata, D., and Rottmann, M. Tars: Traffic-aware radar scene flow estimation. *arXiv preprint arXiv:2503.10210*, 2025.
- Wu, W., Qi, Z., and Fuxin, L. Pointconv: Deep convolutional networks on 3d point clouds. In *CVPR*, pp. 9621–9630, 2019.
- Wu, W., Wang, Z. Y., Li, Z., Liu, W., and Fuxin, L. Pointpwc-net: Cost volume on point clouds for (self-) supervised scene flow estimation. In *Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part V 16*, pp. 88–107. Springer, 2020.
- Yuan, S., Xu, J., Hu, P., Zhu, X., and Zhao, N. Graph smoothing for enhanced local geometry learning in point

cloud analysis. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.

Zhai, M., Bao, B.-K., and Xiang, X. Dmrflow: 4d radar scene flow estimation with decoupled matching and refinement. *IEEE Transactions on Circuits and Systems for Video Technology*, 2025.

Zhang, Q., Yang, Y., Fang, H., Geng, R., and Jensfelt, P. De-flow: decoder of scene flow network in autonomous driving. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2105–2111. IEEE, 2024a.

Zhang, Q., Yang, Y., Li, P., Andersson, O., and Jensfelt, P. Seflow: A self-supervised scene flow method in autonomous driving. In *European Conference on Computer Vision*, pp. 353–369. Springer, 2024b.

Zhao, N., Chua, T.-S., and Lee, G. H. Ps²-net: A locally and globally aware network for point-based semantic segmentation. In *2020 25th International Conference on Pattern Recognition (ICPR)*, pp. 723–730. IEEE, 2021.

Zhou, H., Chang, Y., and Shi, Z. Bring event into rgb and lidar: Hierarchical visual-motion fusion for scene flow. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 26477–26486, 2024.

Zou, X., Yang, J., Zhang, H., Li, F., Li, L., Wang, J., Wang, L., Gao, J., and Lee, Y. J. Segment everything everywhere all at once. *Advances in neural information processing systems*, 36:19769–19782, 2023.

Appendix

This appendix provides more detailed descriptions for input data and data preprocessing in Sec. A. We also show the additional details of our method in Sec. B. Additional experimental results are presented in Sec. C to demonstrate the effectiveness of our method. The discussion of failure cases and limitation analysis is provided in Sec. D.

A. Input Data and Data Preprocessing Details

A.1. Details of VoD dataset

The View-of-Delft (VoD) dataset (Palfy et al., 2022) is collected with a Toyota Prius 2013 platform, which is equipped with a 3D LiDAR, a stereo camera, and a 4D ZF FRGen21 radar in complex urban traffic environments under normal weather conditions. With an annotation frequency of 10 Hz, the VoD dataset consists of 123,106 annotated 3D bounding boxes for both moving and static objects, including 26,587 pedestrians, 10,800 cyclists and 26,949 cars. While the LiDAR point clouds cover a 360 degree range, the tracking annotation and radar data are only available within the viewing frustum. For radar data, the VoD dataset provides three versions of 4D radar point clouds: single-scan, three-scan, and five-scan. For a fair comparison with previous methods (Ding et al., 2022; 2023; Zhai et al., 2025; Wu et al., 2025), we select the single-scan version as input, with an average of 247 radar points per frame. Moreover, every raw radar point in the VoD dataset contains seven aspects of information: 3D location (x, y, z) , reflectivity (RCS), relative radial velocity v_r , absolute radial velocity v_{rc} and scan id τ . We only use the information from the first five dimensions of each radar point as network input:

$$[x, y, z, \text{RCS}, v_r]. \quad (10)$$

A.2. Rigid Scene Flow Annotations from Labeled Tracking Boxes

Since pointwise scene flow annotation is impractical on large-scale real-world datasets, generating scene flow labels from annotated 3D tracking boxes is commonly used in previous studies (Jund et al., 2021; Zhang et al., 2024a; Kim et al., 2025; Khoche et al., 2025; Luo et al., 2025; Zhang et al., 2024b; Ding et al., 2023; Zhai et al., 2025; Wu et al., 2025). Given two annotated tracking boxes with the same track ID in two consecutive frames t_0 and t_1 , the homogeneous transformation $\mathbf{T}$ between these two bounding boxes can be directly inferred. The time interval between the two frames is represented as $\Delta t = t_1 - t_0$. For a point $\mathbf{P}$ in the tracking box at t_0 , its rigid scene flow $\mathbf{F}_r$ is calculated as follows:

$$\mathbf{F}_r = \frac{1}{\Delta t}(\mathbf{T} \otimes \mathbf{P} - \mathbf{P}), \quad (11)$$

where $\otimes$ denotes matrix multiplication. Note that the produced rigid scene flow vector is an approximation of the actual scene flow ground truth, which may not be accurate for non-rigid objects. However, this non-rigid deformation is minimized with a high frame rate.

B. Supplementary Details of Our Method

B.1. Details of Instance Label Generation for Radar Points

Firstly, the images at time t and $t+1$ are fed into a pretrained off-the-shelf 2D multi-object tracking network (Khanam & Hussain, 2024; Gallagher & Oughton, 2025) to generate tracked 2D bounding boxes. Inspired by recent advances in Vision Foundation Models (VFM) (Zou et al., 2023; Kirillov et al., 2023; Ravi et al., 2024), the 2D tracking results can be further refined into more detailed instance-level masks that are aligned with object boundaries. In this way, final instance segmentation masks with track ID can be generated.

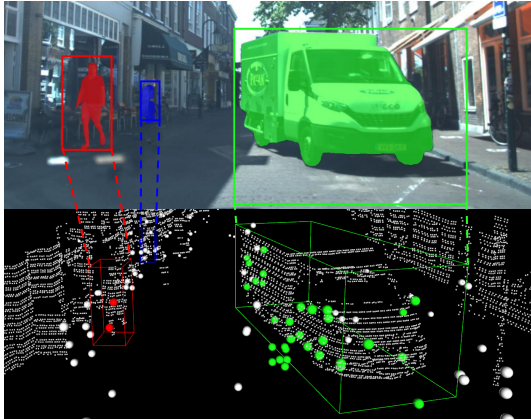


Figure 6. A schematic diagram of obtaining 3D pointwise instance labels from 2D tracking and semantic segmentation results. The smaller points are LiDAR points for auxiliary visualization, while the larger balls are radar points. Each valid radar point associated with a certain instance is painted with the same color as its corresponding 2D tracking box and instance mask.

Given the intrinsic matrix Γ_I of the camera and the extrinsic transformation matrix $\Gamma_{r \rightarrow c}$ between the radar’s and camera’s coordinate system, 3D radar points can be projected onto the 2D image plane:

$$[u, v, 1]^T = \frac{1}{z} \times \Gamma_I \times \Gamma_{r \rightarrow c} \times [x, y, z]^T, \quad (12)$$

where x, y, z are the 3D spatial position of each radar point and u, v is the corresponding 2D pixel coordinate. Each radar point is thereby mapped to the image plane to be associated with a tracked instance or the background and assigned with corresponding instance label or background label. Fig. 6 shows an example of retrieving instance labels for radar points based on the 3D-to-2D correspondence.

B.2. Hyperparameter Sensitivity Analysis.

As shown in Fig. 7, we conduct experiments on the number of scene flow iterations steps K , and ball query hyperparameters, including the search radius R and the number of sampled points L . As K increases, the performance of the network will improve until it approaches convergence, and the time consumption will also increase accordingly. Considering the trade-off between accuracy and efficiency, we set $K = 12$. As for R and L , the accuracy of scene flow estimation varies with different hyperparameter configurations. We select the parameter combination that best optimizes network performance: $R = 1m$ and $L = 8$, as described in Sec. 4.

C. Additional Experiments and Analysis

C.1. Three-way Endpoint Error Evaluation and Runtime Comparison

To comprehensively evaluate the performance of state-of-the-art scene flow estimation methods and our IterFlow, we adopt the three-way Endpoint Error (3-way EPE) for additional evaluation, which is often used in LiDAR-based cases (Jund et al., 2021; Zhang et al., 2024a;b; Khoche et al., 2025; Kim et al., 2025). The 3-way EPE computes the unweighted average EPE of points located in the Foreground Dynamic ($\mathcal{FD}$), Background Static ($\mathcal{BS}$) and Foreground Static ($\mathcal{FS}$) regions. The commonly used 3D end-point-error (3D EPE) results are also presented for comparison.

Note that existing cross-modal supervised methods require additional supervision from dense 3D LiDAR point clouds, 2D optical flow, and ego vehicle’s odometry; and fully-supervised methods use scene flow ground truth for training. The remaining methods (including ours) are free of such direct 3D motion supervision. As shown in Table 5, although our IterFlow is only trained with three 3D label-free losses, it still surpasses cross-modal supervised CMFlow (Ding et al., 2023) on 3-way EPE metrics by a large margin, and even outperforms a series of fully-supervised models. This expanded experimental analysis further proves the effectiveness of our network and loss designs.

Evaluated on a single NVIDIA A40 GPU, the time required for all methods to process a pair of input radar frames is recorded in Table 5. As described in Sec. 4 and Sec. B.2, our best IterFlow sets the number of scene flow iterations steps to $K = 12$ for both network training and inference, which still maintains good time efficiency, as shown in the runtime column of Table 5. We also provide the evaluation results of a lower version of IterFlow, which sets the number of iteration steps as $K = 4$, denoted as IterFlow[†] in Table 5. It can be seen that the runtime required for our IterFlow[†] is similar as CMFlow (Ding et al., 2023), and both

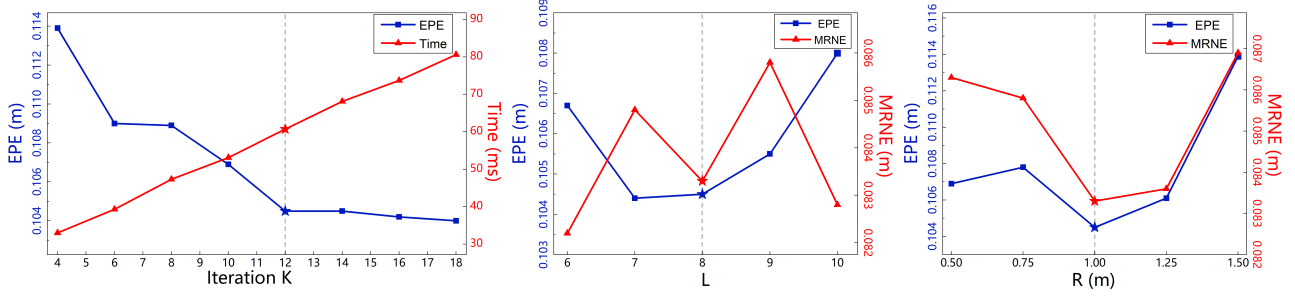


Figure 7. Ablation on iteration steps K and ball query hyperparameters L and R. When L varies, R = 1m; when R varies, L = 8.

Table 5. **3-way EPE Evaluation and Runtime Comparison on VoD validation set.** IterFlow[†] is a lower version of our IterFlow, which uses K = 4 scene flow iterations during both training and evaluation instead of the original setting of K = 12. The (3D Label-free) column indicates whether the network require pseudo 3D scene flow labels derived from dense 3D LiDAR point clouds with off-the-shelf 3D tracking models or ground-truth 3D tracking boxes from the dataset.

Category	Method	Sup.	3D Label Free	Radar					Time (ms)
				Endpoint Error (↓)					
				3-way	3D	FD	BS	FS	
LiDAR- based	Flow4D (Kim et al., 2025)	Fully	✗	0.2247	0.2555	0.2947	0.2506	0.1288	82.0
	BiFlow (Cheng & Ko, 2022)	Fully	✗	0.2015	0.2140	0.3056	0.2067	0.0922	77.2
	DeFlow (Zhang et al., 2024a)	Fully	✗	0.1802	0.2015	0.2422	0.1966	0.1020	44.8
	PointPWC (Wu et al., 2020)	Fully	✗	0.1298	0.1271	0.2125	0.1193	0.0577	53.2
	PV-RAFT (Wei et al., 2021)	Fully	✗	0.1238	0.1140	0.2126	0.1044	0.0545	86.9
	PointPWC (Wu et al., 2020)	Self	✓	0.3560	0.4314	0.4203	0.4333	0.2145	52.1
	FlowStep3D (Kittenplon et al., 2021)	Self	✓	0.2296	0.2607	0.3116	0.2562	0.1210	58.9
radar- based	RaFlow (Ding et al., 2022)	Self	✓	0.2445	0.2753	0.3391	0.2712	0.1233	28.9
	CMFlow (Ding et al., 2023)	Cross	✗	0.1455	0.1600	0.2073	0.1566	0.0727	29.1
	IterFlow [†] (ours)	Cross	✓	0.1223	0.1139	0.2129	0.1049	0.0491	33.0
	IterFlow (ours)	Cross	✓	0.1156	0.1045	0.2058	0.0952	0.0458	60.7

Table 6. Per-category Performance Disparities for FD Objects on VoD validation set.

Cat.	Method	Sup.	Speed Normalized EPE (↓)				
			Mean	Car	O. V.	Pd.	W. V
LiDAR-based	Flow4D (Kim et al., 2025)	Fully	1.3979	1.1186	1.6613	1.4579	1.3538
	BiFlow (Cheng & Ko, 2022)	Fully	1.1973	1.1842	1.1279	1.2416	1.2356
	DeFlow (Zhang et al., 2024a)	Fully	1.1508	0.9907	1.3374	1.2063	1.0690
	PointPWC (Wu et al., 2020)	Fully	0.9919	0.9154	1.2938	0.9768	0.7817
	PV-RAFT (Wei et al., 2021)	Fully	0.9306	0.8949	1.0119	1.0117	0.8041
	PointPWC (Wu et al., 2020)	Self	1.8967	1.3779	2.2572	2.0292	1.9226
	FlowStep3D (Kittenplon et al., 2021)	Self	1.2397	1.1386	1.0888	1.3335	1.3981
Radar-based	RaFlow (Ding et al., 2022)	Self	1.4237	1.3608	1.4364	1.4597	1.4378
	CMFlow (Ding et al., 2023)	Cross	0.9676	0.9029	1.0947	1.0396	0.8332
	IterFlow (ours)	Cross	0.8848	0.8193	0.8275	1.0044	0.8877

methods have achieved real-time performance; however, our IterFlow[†] outperforms CMFlow on the 3-way EPE and 3D EPE metrics, with performance improvements of 15.9% and 28.8%, respectively. The experimental results in Table 5 show that IterFlow[†] brings a significant performance improvement in the static areas, but the EPE in the foreground dynamic (FD) region increases slightly due to the lack of direct 3D motion supervision and the limited iteration steps.

C.2. Foreground Category Performance Analysis

We adopt the Bucketed Normalized EPE metric (Khatri et al., 2024) to ensure that objects with different speeds are fairly evaluated. The dynamic normalized EPE is a ratio as the EPE has been normalized by speed. As shown in Table 6, fine-grained analysis is conducted on individual classes, including Car, Other Vehicles (O. V.), Pedestrian

Table 7. Quantitative Evaluation on MAN TruckScenes val set. In the Category (Cat.) column, existing methods are classified depending on the input modality used in their original work. In the Supervision (Sup.) column, the methods are categorized as fully supervised (Full), self-supervised (Self), or cross-supervised (Cross). * denotes using the ground truth 3D track IDs instead of pointwise instance IDs derived from 2D images for instance-aware loss calculation.

Cat.	Method	Sup.	Overall				Moving	Static
			EPE [m]↓	AccS [%]↑	AccR [%]↑	RNE [m]↓	MRNE [m]↓	SRNE [m]↓
LiDAR-based	Flow4D (Kim et al., 2025)	Full	0.2368	19.07	51.93	0.0934	0.1077	0.0866
	DeFlow (Zhang et al., 2024a)	Full	0.1984	38.13	63.48	0.0783	0.0748	0.0745
	PointPWC (Wu et al., 2020)	Full	0.1151	70.63	86.23	0.0454	0.0639	0.0413
Radar-based	RaFlow (Ding et al., 2022)	Self	0.4537	2.59	13.43	0.1789	0.1334	0.1710
	CMFlow (Ding et al., 2023)	Full&Cross	0.1342	38.20	73.17	0.0531	0.0253	0.0522
	IterFlow (ours)	Cross	0.1103	64.30	83.00	0.0435	0.0680	0.0389
	IterFlow* (ours)	Cross	0.0996	68.53	84.65	0.0393	0.0673	0.0342

(Pd), and Wheeled VRU (W. V.) on VoD validation dataset. The experimental results indicate that in traffic scenarios, small targets with low-speed (*e.g.* pedestrians) are more challenging for scene flow estimation than other kinds of objects.

C.3. Supplementary Experimental Results on MAN TruckScenes Dataset

To broaden the evaluation scope, we conduct supplementary experiments on the latest MAN TruckScenes dataset, which contains 747 scenes under diverse environmental conditions, including fog, rain, snow, daytime, and nighttime. Note that MAN TruckScenes provides radar data sampled at 10 Hz but manual annotations only at 2 Hz, so intermediate sweeps are unlabeled and cannot be directly used for standard scene flow evaluation. Since ground truth scene flow is not provided in the original dataset, we generate scene flow labels based on the annotated tracking boxes for the keyframes in its official training and validation splits.

As shown in Table 7, all scene flow estimation methods are trained from scratch on the MAN TruckScenes training set and evaluate on the validation set. To train CMFlow (Ding et al., 2023) on the MAN TruckScenes dataset, we generate extra required 2D optical flow labels by adopting their officially released pretrained 2D optical flow estimation model and we directly adopt ground truth 3D scene flow labels to provide full supervision for CMFlow’s training, since no 3D tracking results from pretrained model on the MAN TruckScenes dataset are provided. The training settings of other methods on the MAN TruckScenes dataset remain the same as their original settings on the VoD dataset. Experimental results show that our method still achieves the best performance on overall EPE metric and obtains performance comparable to fully supervised methods on the challenging MAN TruckScenes dataset, which demonstrates the effectiveness of our design. Moreover, compared to the last two rows of Table 3, the performance gap between IterFlow* and IterFlow on the MAN TruckScenes dataset is more pronounced than that on the VoD dataset. This is because unlike the VoD dataset whose weather conditions are

generally good, MAN TruckScenes covers a wide range of environmental conditions, making it more difficult for 2D models to generate accurate instance labels during training.

Table 8. Detection Performance of different versions of YOLO11 Model on the COCO validation set.

Model	mAP 50-95	Runtime (ms)	params (M)	FLOPs (B)
YOLO11m	51.5	4.7	20.1	68.0
YOLO11l	53.4	6.2	25.3	86.9
YOLO11x	54.7	11.3	56.9	194.9

Table 9. 2D Zero-shot Segmentation Performance of SAM with different scales of Vision Transformer(ViT) backbone on the COCO dataset.

Backbone Version	mIoU (%)	mAP @0.5	mAP @0.75	FPS	Params (B)
ViT-B (Base)	74.3	78.2	71.6	22.2	375M
ViT-L (Large)	76.8	80.9	74.5	12.8	1.25G
ViT-H (Huge)	78.2	82.5	76.8	8.0	2.56G

C.4. Supplementary Experimental Results on Various Versions of 2D Tracking and Segmentation Models

As described in Sec. 4, we use the officially released YOLO11-l (Khanam & Hussain, 2024) model and SAM (Kirillov et al., 2023) with huge version of Vision Transformer (ViT-H) backbone for 2D instance label generation in the main experimental section of this paper. As shown in Table 8 and Table 9, the official repository of YOLO11 and SAM also provide various versions of pretrained models with different parameter size. The YOLO11 models are trained on the COCO dataset (Lin et al., 2014) and the SAM models are trained on their SA-1B dataset, and more details are given in their official documents.

As shown in Table 10, we provide supplementary scene flow estimation results of IterFlow with 2D instance-level guidance from different combinations of off-the-shelf 2D

Table 10. Comparison of IterFlow’s Performance with 2D semantic guidance from different versions of off-the-shelf 2D tracking and segmentation models on VoD validation set.

YOLO11 Model	SAM Model	Overall				Moving	Static
		EPE [m]↓	AccS [%]↑	AccR [%]↑	RNE [m]↓	MRNE [m]↓	SRNE [m]↓
YOLO11m	H	0.1025	33.90	64.91	0.0413	0.0850	0.0369
YOLO11l	H	0.1045	33.40	63.75	0.0420	0.0833	0.0380
YOLO11x	H	0.1031	34.42	65.46	0.0415	0.0877	0.0371
YOLO11l	B	0.1039	33.78	64.46	0.0418	0.0880	0.0374
YOLO11l	L	0.1040	33.22	64.29	0.0419	0.0883	0.0373
YOLO11l	H	0.1045	33.40	63.75	0.0420	0.0833	0.0380

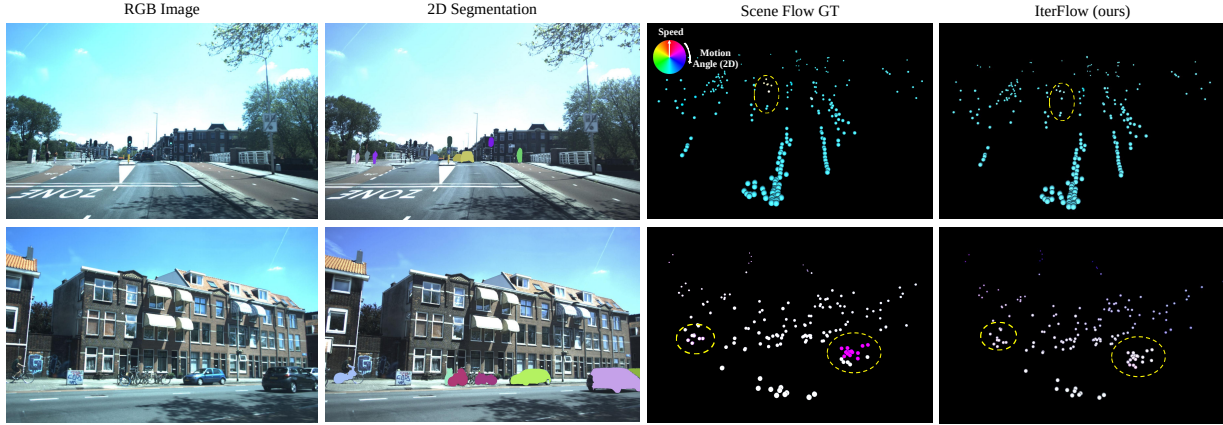


Figure 8. Visualization of failure cases on VoD validation set. Each row displays a driving scenario and regions with large scene flow estimation errors are highlighted with yellow circles.

models. It can also be observed that current commonly used 2D models can effectively provide weak supervision signals for our 4D radar scene flow estimation method and the network performance is generally robust as the combination of 2D models varies.

D. Failure Cases and Limitation Analysis

As shown in Fig. 8, we provide qualitative visualization results of failure cases for our method on VoD validation set. It can be observed that although reliable 2D instance segmentation masks have been provided by off-the-shelf 2D models, the network may fail in difficult situations, which indicates that our method is mainly limited by the sparse and noisy nature of radar data itself, rather than insufficient support from 2D semantics.

In the scene of the first row, the network incorrectly estimates the scene flow of a small number of sparse points on distant pedestrians. This is because the radar points are extremely sparse in distant areas and very few valid points are associated with difficult dynamic foreground objects; In this case, very small noise may prevent these foreground radar points from being projected onto the correct pixels in the 2D image, further impacting the optimization of predicted

flows in such challenging situations.

In another scene below, the network fails to estimate the scene flows on the left cyclist and the right vehicle. Handling the large displacements of dynamic foreground objects between frames is naturally hard for scene flow estimation tasks, since the cross-frame feature correlation for flow embedding usually relies on nearest neighbor search between the point clouds. Such challenges become more prominent when it comes to radar scene flow estimation in real-world traffic scenarios, because the lack of reliable correspondences for matching instance pairs across frames and radar data is more sensitive to abnormal deformations and random noise due to the sparsity.